

Word_Cloud_Each_Sample

2026-02-17

Data Under Consideration:

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

## Unique_Panelist_ID Sample
## 1          441683      H
## 2          441683      F
## 3          441683      B
## 4          441683      K
## 5          441683      C
## 6          441683      I
## 7          441683      J
## 8          441683      D
## 9          441683      G
## 10         441683      M

##                               Comments_aroma_liking
## 1      it smells like parmesan, nice distinct smell
## 2                               cheesey and not too harsh
## 3                                       n/a
## 4                               earthy, mild
## 5      buttery, woody. made me salivate a little
## 6      that buttery smell when i first smelled it
## 7                                       n/a
## 8      it doesnt smell bad, it just doesnt smell good
## 9      cheesey but not a punch to the face, kind of earthy
## 10 like a mix of parmesan and cheddar, earthy but sweet

##                               Comments_aroma_disliking
## 1                                       n/a
## 2      really smelled sour for a bit
## 3      smelled like cardboard
## 4      faint sour smell
## 5      woodsy. it smells good but not what i want in my food.
## 6 it somehow ended up smelling like nail polish remover the longer i smelled it
## 7      it is a really harsh smell. gave me a small gag reflex
## 8      smells too woodsy and a little like cardboard
## 9                                       n/a
## 10                                      n/a
```

Cleaning the text data:

1. Rename columns for easier coding
2. Recode things that look like NAs or null values to NA
3. Drop the rows where both positive and negative reviews are NA or null
4. Unpivot the data each customer x sample combination will have 2 rows one for positive and one for negative review (need this for the way I created the wordclouds)
5. Tokenize the words and remove stop words
6. Count the frequency of words (by sample and sentiment) and then calculate TF-Idf (Term frequency Inverse Document frequency)

TF-Idf penalizes generic words that are frequently occurring in reviews for other samples

TF = Probability (proportion) of seeing a word in a document (one sample X sentiment combination) $IDF = \log(N/df_w)$,

N = the total number of documents (total sample x sentiment combinations),

df_w = number of documents containing the given word

$TF-idf = TF * IDF = P(\text{observing given word in your document}) * \text{how often the word appears overall}$

Intuition: We are penalizing common words, we want words that correlate with specific cheese to have high values.

Unpivot for comparisons and easier computations, remove stop words, tokenize and calculate frequency

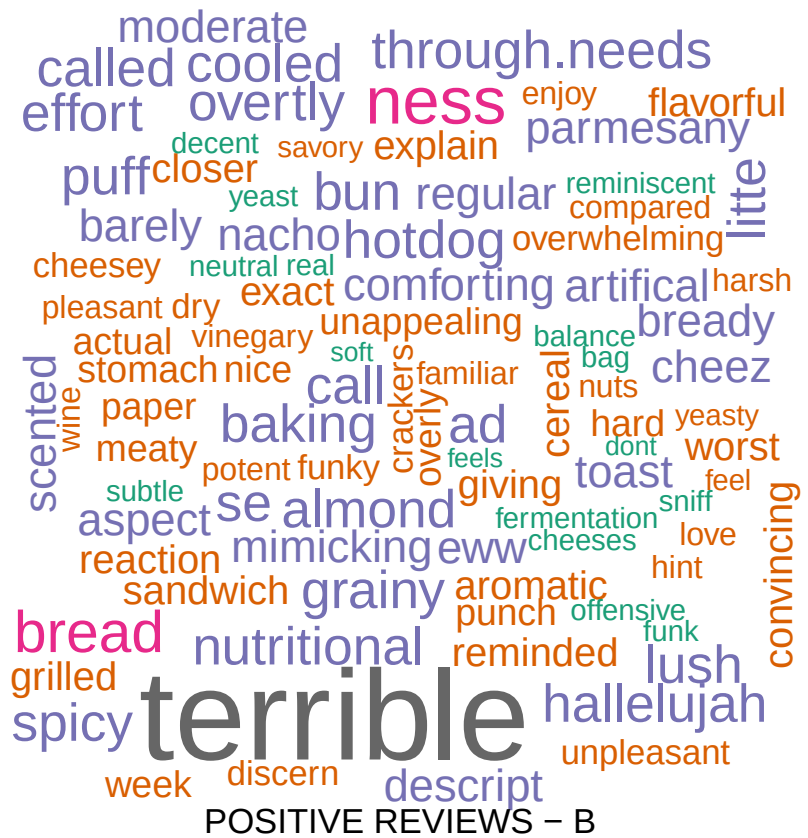
Function to create wordclouds given Sample and whether you want positive or negative review takes `ch` and `pos` as input.

`ch`: Sample Name (A, B, C,...)

`pos`: positive or negative.

Set `ch` = sample (A, B, C,...) and `pos` = "positive" if you'd like a wordcloud for positive review and "negative" if you'd like one for negative reviews.

Sample A:



Positive Reviews:



Positive Reviews:

POSITIVE REVIEWS – C



NEGATIVE REVIEWS – C

Negative Reviews:

Sample D:



NEGATIVE REVIEWS – F

Negative Reviews:

Sample G:



Positive Reviews:

POSITIVE REVIEWS – H



Negative Reviews:

Sample I:



Positive Reviews:



Negative Reviews:

Sample L:

disrupting overwhelming
 soft 50000 inoffensive decent
 balanced spoiled
 gently passes smoky
 passable bring
 popcorn reminds purchase oily
 acid level
 smoked clean jar
 stick greatly eaten
 difference touch mild
 chessey toned white
 overpower warm nice
 resembles dairyish
 reminded fan

POSITIVE REVIEWS - L



NEGATIVE REVIEWS – M

Negative Reviews: